and deliver a suitable smart contract solution. The smart contract syntax is similar to that of class in object-oriented programming, with the additional caveat that especially careful design is required for the trust and integrity elements.

You've also learned several special techniques for realizing these trust elements, including implementing trust intermediation through modifiers that enable validation and verification of conditions in a smart contract. Modifiers can also support privacy, security, confidentiality, and (thus) integrity by managing access to your data and functions.

3.8 Summary

- Trust and integrity are critical needs in a decentralized system, in which the participants operate beyond traditional boundaries of trust. In a decentralized system, no humans are checking your credentials, such as a driver's license, and no system is verifying your username/password combination for authentication.
- Trust in blockchain-based application development is achieved by verification and validation through a trio of features: modifiers, `require()`, and `assert()`.
- The `revert()` declaration reverts a function call and prevents transactions from being recorded in the blockchain's immutable ledger, thus preventing invalid information from accumulating in the ledger.
- FSM design provides another important design diagram, especially for the design of smart contracts with state transitions.